



EÖTVÖS LORÁND TUDOMÁNYEGYETEM

INFORMATIKAI KAR

PROGRAMOZÁSI NYELVEK ÉS FORDÍTÓPROGRAMOK TANSZÉK

Smart Turntable Assistant

Témavezető:

Bense Viktor

Doktorandusz

Szerző:

Kiss Dániel

programtervező informatikus BSc

Budapest, 2026

Tartalomjegyzék

1. Bevezetés	3
1.1. Témaválasztás indoklása	3
1.2. A megoldandó feladat leírása	4
1.3. A szakdolgozat célkitűzései	4
2. Felhasználói dokumentáció	6
2.1. A megoldott probléma megfogalmazása	6
2.2. A felhasznált módszerek rövid leírása	6
2.3. A program használatához szükséges információk	7
2.3.1. Telepítés és Indítás	7
2.3.2. Alapvető Funkciók Használata	7
2.3.3. Külső Integrációk	8
3. Fejlesztői dokumentáció	9
3.1. Követelményelemzés és specifikáció	9
3.1.1. Funkcionális követelmények	9
3.1.2. Nem-funkcionális követelmények	10
3.1.3. Használati esetek (Use Cases)	10
3.2. Felhasznált technológiák és módszerek	11
3.2.1. Audio Fingerprinting technológiák	11
3.2.2. Alkalmazott keretrendszerek és futtatási környezet	11
3.3. Szoftver architektúra és szerkezet	12
3.3.1. Adatbázis Modell	12
3.3.2. Konkurencia és szálkezelés	13
3.4. Megvalósítási részletek	14
3.4.1. Digitális jelfeldolgozás (DSP)	14
3.4.2. A vízesés elvű metaadat-stratégia	14
3.4.3. Frontend renderelés és szinkronizáció	14

3.5. Tesztelés és Értékelés	14
3.5.1. Manuális és integrációs tesztelés	14
3.5.2. Teljesítmény-optimalizáció	15
3.6. Mesterséges intelligencia alkalmazása a fejlesztés során	15
4. Összegzés és jövőbeli fejlesztések	17
4.1. Az elért eredmények értékelése	17
4.2. Jövőbeli fejlesztési lehetőségek	17
Ábrajegyzék	19

1. fejezet

Bevezetés

1.1. Témaválasztás indoklása

Az elmúlt években a zeneipar az analóg fizikai hordozók, különösen a bakelit-lemezek népszerűségének példátlan visszatérését tapasztalta. Az audiofileket és a hétköznapi embereket egyaránt vonzza a mechanikus élmény, a szándékos cselekvés, amellyel egy albumot az elejétől a végéig meghallgatnak, valamint az analóg hangzás egyedi, meleg akusztikai jellemzője. Ugyanakkor egy olyan korszakban, amelyet a digitális streaming platformok, mint a Spotify és az Apple Music uralnak, a hagyományos hanglemez hallgatási élmény alapvetően elszigetelt marad a modern digitális ökoszisztémától.

A témát azért választottam, mert létezik egy jelentős technológiai szakadék az analóg audio jel és a modern metaadat-kezelés között. Egy olyan eszköz létrehozása, amely képes hidat képezni a fizikai lemezjátszó és a digitális kényelmi funkciók között, nemcsak technikai kihívást jelent a valós idejű jelfeldolgozás terén, hanem valódi hozzáadott értéket nyújt a zenehallgatók számára, akik nem szeretnék lemondni az analóg élményről, de igényelnék a digitális kor előnyeit.

Ez a szakdolgozat egy Smart Turntable Assistant (Okos Lemezjátszó Asszisztens) tervezését, architektúráját és megvalósítását mutatja be, amely egy szoftvervezérelt hidat képez az analóg audio hardverek és a modern digitális kényelmi funkciók között. Valós idejű jelfeldolgozás és külső API integrációk felhasználásával ez a projekt arra törekszik, hogy modernizálja az analóg zenehallgatás élményét anélkül, hogy a bakelit-lemez-hallgatás élményébe, hangvilágába beleszólna.

1.2. A megoldandó feladat leírása

A projekt célja egy olyan „Okos Lemezjátszó Asszisztens” szoftveres rendszer megvalósítása, amely passzívan figyeli egy szabványos lemezjátszó audio kimenetét, és automatikusan nyújt digitálisan kiegészítéseket a hallgatáshoz.

A rendszernek a következő alapvető feladatokat kell megoldania:

- **Automatikus felismerés:** A bejövő analóg jel alapján meg kell határozni, hogy egy zeneszám elkezdődött-e, majd audio fingerprinting segítségével azonosítania kell a pontos előadót és dalcímet.
- **Metaadat-szinkronizáció:** A felismerést követően a rendszernek le kell kérdeznie a számhoz tartozó albumborítót és a szinkronizált dalszöveget, majd azt valós időben meg kell jelenítenie a felhasználónak.
- **Hardveres monitorozás:** A szoftvernek matematikai elemzéssel nyomon kell követnie a hangszedő (tű) kopását a lejátszási órák alapján, valamint diagnosztikai adatokat (pl. motorrezgés, felületi zaj) kell szolgáltatnia.
- **Digitális naplózás:** A hallgatási élményt automatikusan szinkronizálnia kell külső platformokkal (pl. Last.fm), így az analóg lejátszás is digitális nyomon követhetővé válik.

1.3. A szakdolgozat célkitűzései

A szakdolgozat elsődleges célja egy robusztus, valós idejű Kliens-Szerver alkalmazás megtervezése és megvalósítása, amely passzívan figyeli egy szabványos lemezjátszó audio kimenetét, és „okos” funkciók átfogó készletét nyújtja. A rendszert úgy terveztem, hogy egy USB audio interfészhez csatlakoztatott, alacsony erőforrás-igényű hardveren fusson.

A fent vázolt problémák megoldása érdekében a projekt a következő műszaki célkitűzéseket határozza meg:

- **Valós idejű jelfeldolgozás:** Egy konkurens Python backend fejlesztése, amely képes az analóg audio stream folyamatos elemzésére egy zeneszám kezdetének vagy végének detektálásához, valamint a hardverdiagnosztikai adatok matematikai kiszámításához.

- **Audio Fingerprinting és metaadat-bővítés:** Egy robusztus felismerési pipeline implementálása, amely rövid hangmintákat rögzít, és audio fingerprinting szolgáltatások segítségével azonosítja azokat. Továbbá egy waterfall elvű metaadat-lekérdező rendszer tervezése, amely megbízhatóan lekéri a pontos sávhosszokat, albumneveket és nagy felbontású borítóképeket.
- **Automatizált digitális szinkronizáció:** Háttér szolgáltatások létrehozása az időbélyeggel ellátott dalszövegek letöltésére, valamint egy automatizált Last.fm scrobbling funkció implementálása, amely pontosan tiszteletben tartja a sáv eltelt lejátszási idejét.
- **Adatbázis- és hardverkezelés:** Egy relációs adatbázis megvalósítása a hallgatási előzmények tárolására, a testreszabott vizuális preferenciák mentésére, valamint a felhasználó által definiált lemezjátszó-hangszedők életciklusának pontos nyomon követésére.
- **Modern, reaktív felhasználói felület:** Egy reszponzív Single Page Application (SPA) tervezése Vue.js használatával, amely WebSockets technológián keresztül kommunikál a backenddel. Az interfésznek zero-latency vizuális visszajelzést kell nyújtania a felhasználók számára, és dinamikus tematikus testreszabhatóságot kell kínálnia.

2. fejezet

Felhasználói dokumentáció

2.1. A megoldott probléma megfogalmazása

A hagyományos bakelit-lemezjátszó legnagyobb hátránya, hogy a generált jel egy nyers elektromos jel, amely nem tartalmaz metaadatokat. Ez azt jelenti, hogy a felhasználó számára nincs automatikus módja a lejátszott zeneszámok naplózására, nincs hozzáférése a szinkronizált dalszövegekhez, és nincs pontos eszköze a tű kopásának vagy a lemezjátszó technikai állapotának ellenőrzésére. A Smart Turntable Assistant ezen „információs vakpontot” törli el egy intelligens szoftverréteg bevezetésével.

2.2. A felhasznált módszerek rövid leírása

A rendszer egy kliens-szerver architektúrán alapul, ahol a backend és a frontend különálló egységekben működnek:

- **Audio elemzés és azonosítás:** A zeneszámok azonosításához külső Audio Fingerprinting API-kat (pl. Shazam) alkalmaz a rendszer, amelyek egyedi akusztikai „ujjlenyomatokat” hasonlítanak össze egy globális adatbázissal.
- **Valós idejű kommunikáció:** A szerver és a felhasználói felület közötti adatcsere WebSockets (Socket.IO) technológiával történik, ami lehetővé teszi, hogy a diagnosztikai adatok és a dalszövegek késleltetés nélkül érkezzenek a képernyőre.

- **Felhasználói felület:** Egy modern, reaktív weboldal (SPA), amely bármilyen eszközön (mobil, asztali) elérhető és dinamikusan változtatja a megjelenését a lejátszott album borítójához igazodva.
- **Adatkezelés:** Az SQLite adatbázis tárolja a felhasználói beállításokat és a tű élettartamát, így a rendszerbe bejelentkezve a felhasználó mindig a saját konfigurációját kapja vissza.

2.3. A program használatához szükséges információk

2.3.1. Telepítés és Indítás

A program Docker konténerben fut, ami egyszerűsíti a telepítést. A futtatáshoz szükség van egy USB audio interfészre, amelyhez a lemezjátszó csatlakozik.

1. A Docker image letöltése és a konténer indítása a mellékelt `docker-compose.yml` fájlal.
2. A böngészőben a `http://localhost:5000` (vagy a szerver IP címe) megnyitása.

2.3.2. Alapvető Funkciók Használata

- **Zenehallgatás:** Helyezze a tűt a lemezre. A rendszer automatikusan érzékeli a hangot, azonosítja a számot és megjeleníti az albumborítót, valamint a dalszöveget.
- **Hangszedő regisztrálása:** A profil menüben kell megadni az új tű típusát és a gyártó által meghatározott élettartamát. A rendszer ettől kezdve automatikusan számolja a lejátszott órákat.
- **Diagnosztika:** A diagnosztika fülön valós időben követhető a „Rumble” (motorrezgés) és a „Sibilance” (magas frekvenciás torzítás) szintje, valamint a felületi pattogások száma.
- **Testreszabás:** A vizuális beállításoknál módosítható a felület témája, hogy illeszkedjen a fizikai lemez színéhez.

2.3.3. Külső Integrációk

A Last.fm naplózás beállításához a profilba kell lépni, ahol a rendszer egy hitelesítési folyamat után kapcsolódik a felhasználó fiókjához. Ezután minden lejátszott szám automatikusan rögzítésre kerül a profilban.

3. fejezet

Fejlesztői dokumentáció

3.1. Követelményelemzés és specifikáció

A rendszer alapvető célja egy olyan szoftveres megoldás létrehozása, amely képes egy analóg audio jelből zenei metaadatokat és diagnosztikai információkat kinyerni. A funkcionális és nem-funkcionális követelmények meghatározzák azokat a korlátozkat és teljesítménymutatókat, amelyeket az architektúrának ki kell elégítenie.

3.1.1. Funkcionális követelmények

- **Valós idejű audio azonosítás:** A rendszernek folyamatosan figyelnie kell a bejövő audio streamet, és automatikusan detektálnia kell a lejátszás kezdetét. Az észlelés után rögzítenie kell egy rövid audio buffert, és külső audio fingerprinting API-kat kell lekérdeznie a sáv azonosításához.
- **Metaadat-bővítés:** Sikeres audio felismerés után másodlagos adatbázisokat kell lekérdeznie a nagy felbontású albumborítók, a pontos sávhosszúság és a szabványosított albumnév lekéréséhez.
- **Szinkronizált dalszöveg megjelenítés:** A backend-nek le kell töltenie az időbélyeggel ellátott dalszövegfájlokat, a frontend-nek pedig automatikusan kell görgetnie a szöveget, szinkronban az élő lejátszási időzítővel.
- **Hardverállapot-diagnosztika:** A szoftvernek matematikailag fel kell dolgoznia a hangot a hirtelen felületi pattogások detektálására, az alacsony frekvenciás mechanikai motorrezgés mérésére, és a magas frekvenciás sziszegő torzítás (sibilance) számszerűsítésére.

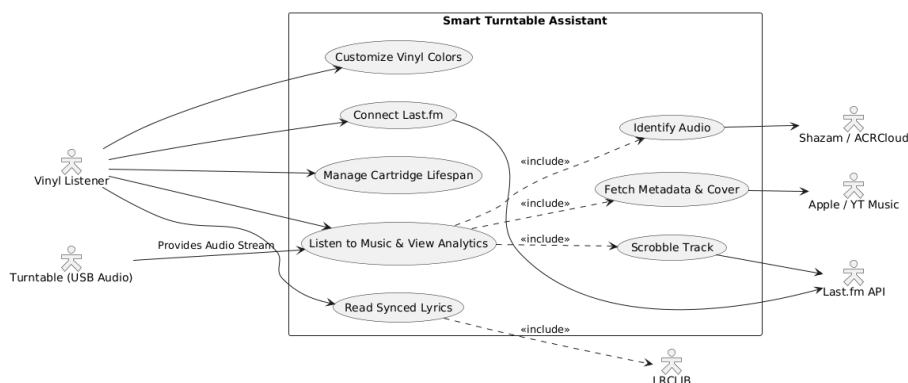
- **Hangszedő életciklus kezelés:** Adatbázis fenntartása a felhasználó lemezjátszó tűiről, nyomon követve az aktív hardver kumulatív lejátszási idejét.
- **Digitális hallgatási naplók:** Hitelesítés és automatikus scrobbling külső platformokra (pl. Last.fm).

3.1.2. Nem-funkcionális követelmények

- **Teljesítmény és késleltetés:** A háttérben futó audiofeldolgozó szálnak folyamatosan működnie kell anélkül, hogy blokkolná a webszerver műveleteit. A diagnosztikai adatokat másodpercenként többször kell a frontend felé továbbítani.
- **Megbízhatóság és Fallback mechanizmusok:** A hálózati lekérdezéseknek robusztus fallback stratégiákat kell alkalmazniuk. Ha egy elsődleges felismerő motor meghibásodik, a rendszernek meg kell próbálnia a másodlagos források használatát.
- **Adatbázis-konkurencia:** A relációs adatbázisnak támogatnia kell az egyidejű olvasási és írásai műveleteket (Write-Ahead Logging módban).

3.1.3. Használati esetek (Use Cases)

A felhasználó és a Smart Turntable Assistant közötti elsődleges interakciókat a 3.1. ábra szemlélteti. A rendszer automatikusan detektálja a zeneszám kezdetét, azonosítja azt, és megjeleníti a szinkronizált dalszöveget. A felhasználó nyomon követheti a diagnosztikát, regisztrálhat új hangszedőt, és testreszabhatja a vizuális felületet a bakelitlemez színének megfelelően.



3.1. ábra. UML Use Case Diagram a felhasználó és a rendszer interakcióiról.

3.2. Felhasznált technológiák és módszerek

Egy valós idejű hardvermonitorozó rendszer fejlesztése a digitális jelfeldolgozástól a modern webalkalmazás-architektúrákig számos tudományág integrációját igényli.

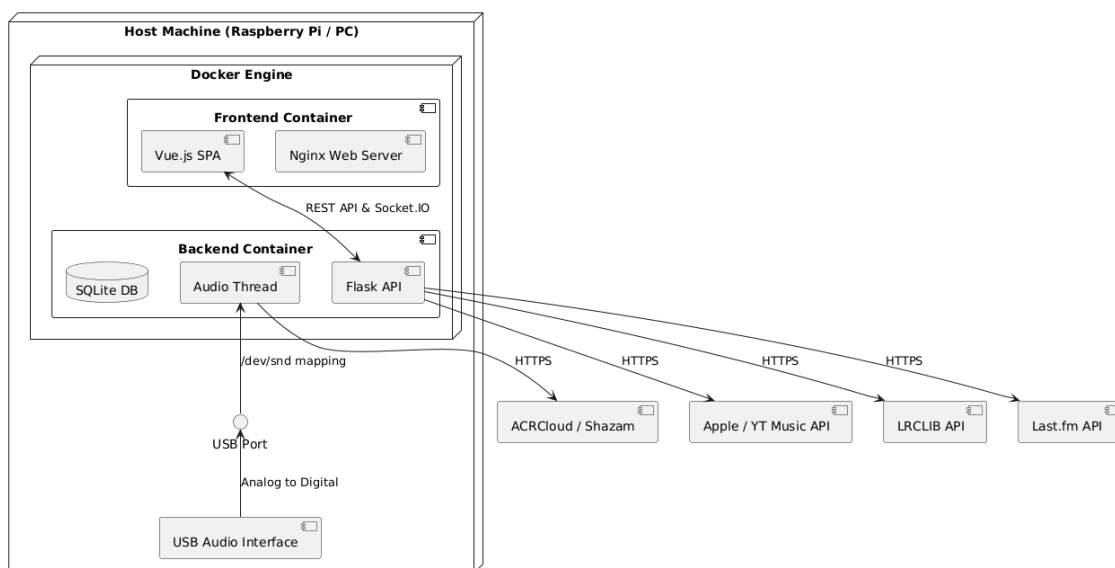
3.2.1. Audio Fingerprinting technológiák

Az audio fingerprinting egy hangminta azonosítására használt mechanizmus. A folyamat jellemzően az audiojel spektrogrammá alakítását, a nagy energiájú pontok azonosítását, és egy constellation map létrehozását foglalja magában, ami rendkívül ellenálló a bakelitlemezekre jellemző felületi sercegéssel szemben. A projektben az ACRCLOUD és a Shazam API kerül felhasználásra a felismerés elvégzésére.

3.2.2. Alkalmazott keretrendszerek és futtatási környezet

A backend Python nyelven készült, mivel kiterjedt ökoszisztémája (NumPy, SciPy) ideális a matematikai számításokhoz. A webszerver réteget a Flask hajtja meg. A frontend egy Vue.js alapú SPA, ahol a globális állapotkezelést a Pinia végzi. A valós idejű (zero-latency) kommunikációt a WebSockets (Socket.IO) biztosítja.

A környezeti izoláció és a hordozhatóság érdekében a rendszert Docker konténerbe csomagoltam, ami lehetővé teszi a hoszt ALSA és PulseAudio hangeszközeinek közvetlen leképezését a konténerbe (lásd 3.2. ábra).



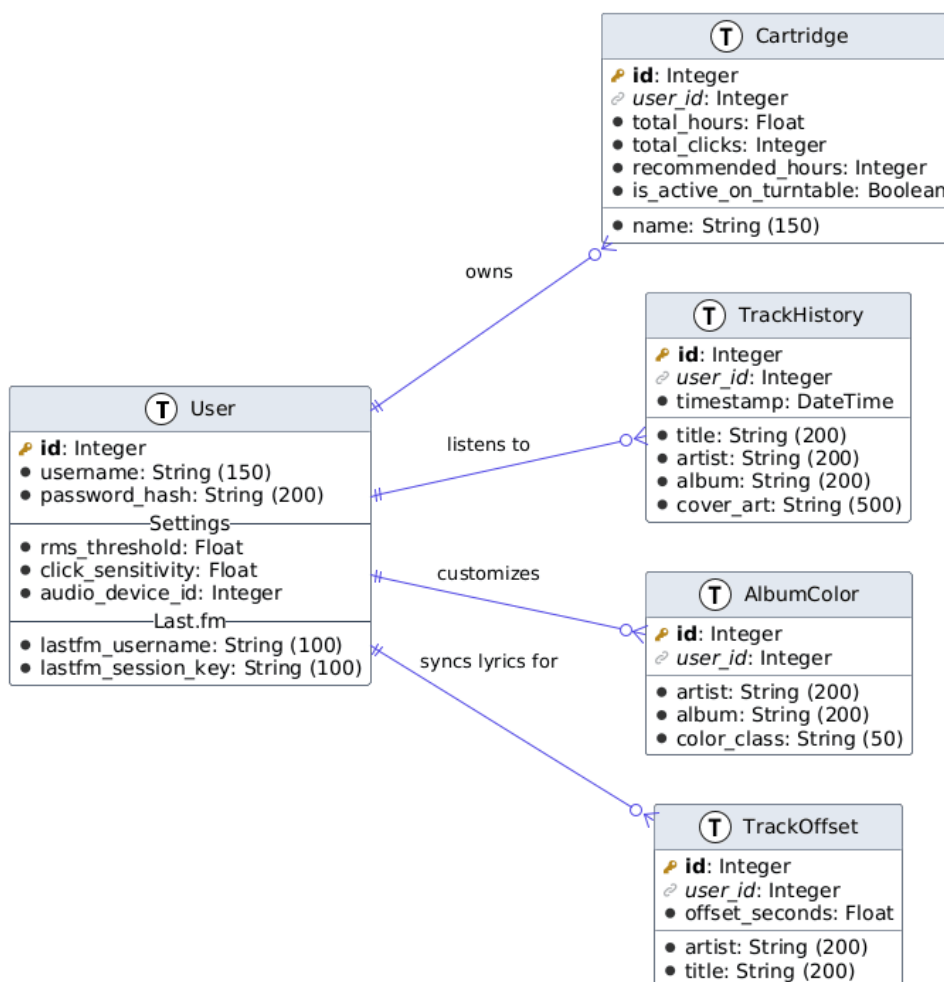
3.2. ábra. Deployment diagram, amely a Docker image felépítését mutatja be.

3.3. Szoftver architektúra és szerkezet

Az alkalmazás egy robusztus kliens-szerver modellt követ. A kommunikáció e két eltérő környezet között kétirányú: az állapotváltozásokat REST API végpontok, míg a telemetriát WebSocket csatornák kezelik.

3.3.1. Adatbázis Modell

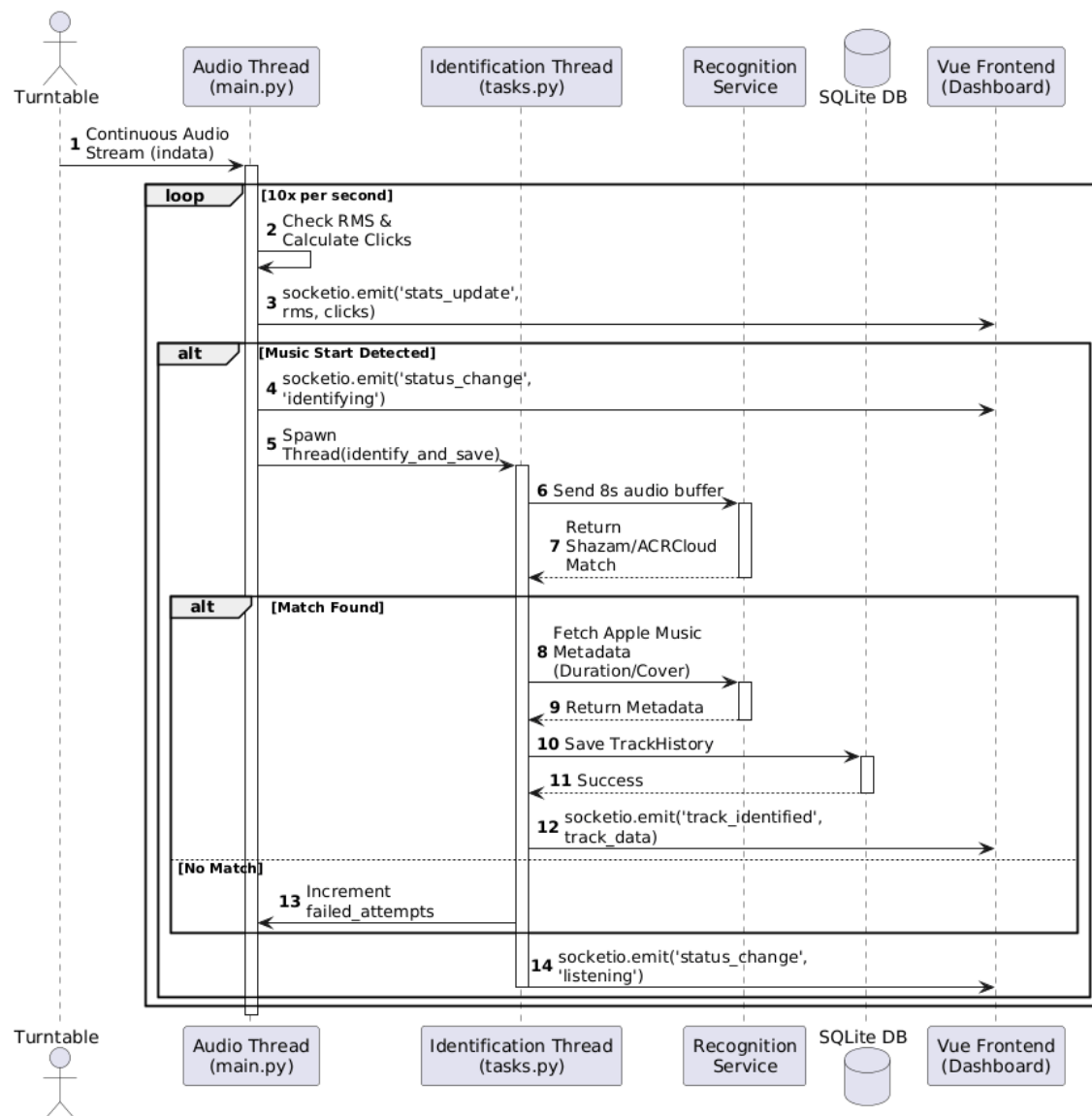
A perzisztens adattárolást SQLite kezeli SQLAlchemy ORM használatával (lásd 3.3. ábra). A **User** entitás biztosítja az adatok izolációját. A **Cartridge** tábla követi a tűk üzemóráit, míg a **TrackHistory** digitális hallgatási naplóként funkcionál. Az esztétikai preferenciákat az **AlbumColor** és **TrackOffset** táblák tárolják.



3.3. ábra. Entity-Relationship Diagram az adatbázis sémájáról.

3.3.2. Konkurencia és szálkezelés

A backend többszálú architektúrát alkalmaz (lásd 3.4. ábra). Az elsődleges szál folyamatosan olvassa az audio buffert, számítja a diagnosztikát, és megállapítja egy új dal kezdetét. Ekkor elindít egy független másodlagos szálat, amely elvégzi a hálózati kéréseket az audio azonosításhoz, elkerülve a UI megfagyását.



3.4. ábra. UML Sequence Diagram a többszálú adatfeldolgozásról.

3.4. Megvalósítási részletek

3.4.1. Digitális jelfeldolgozás (DSP)

Annak detektálására, hogy a felhasználó mikor teszi rá a tűt a lemezre, a rendszer kiszámítja az audio chunk RMS értékét. A felületi tökéletlenségek (clicks or pops) detektálásához a szoftver kiszámítja az audiojel első deriváltját, amely felül áteresztő szűrőként (high-pass filter) funkcionál. Ezen változások szórásának kiszámításával az algoritmus statisztikai kiugró értékeket keres. A rumble kiszámításához Fast Fourier Transzformációt (FFT) alkalmazok a 10Hz-50Hz közötti energia összegzéséhez, a sibilance-t pedig egy Butterworth sávszűrővel (5kHz-10kHz) határozom meg.

3.4.2. A vízesés elvű metaadat-stratégia

A hiányos információk elkerülése érdekében az alkalmazás egy cascading waterfall stratégiát alkalmaz. Az elsődleges fingerprinting API válasza alapján a backend lekérdezi az iTunes webszolgáltatást a nagy felbontású borítóképekért. Ha ez meghiúsul, a rendszer visszalép egy szövegalapú keresésre a `ytmusicapi` segítségével, és dinamikusan manipulálja a YouTube Music kép URL paramétereit a nagy felbontás garantálása érdekében.

3.4.3. Frontend renderelés és szinkronizáció

A Vue.js felület reszponzivitását CSS konténer-lekérdezések és dinamikus adat-attribútumok (témázás) biztosítják. Szinkronizált dalszöveg esetén a frontend a letöltött időbélyegeket folyamatosan összehasonlítja a backend által közvetített élő lejátszási időzítővel, és a Vue `nextTick` renderelési ciklusát használja az aktív sor precíz görgetéséhez. A Last.fm scrobbling-ot egy háttérszál végzi, miután az eltelt sávidő eléri az 50%-ot.

3.5. Tesztelés és Értékelés

3.5.1. Manuális és integrációs tesztelés

A rendszert különféle fizikai bakelitlemezekkel (eltérő műfajok és felületi állapotok) értékeltük. Ismert fizikai sérülésekkel rendelkező lemezeket játszottunk le annak

megerősítésére, hogy a pattogás detektáló (click detection) logika nem ad fals pozitív találatokat agresszív dob ütéseknel. A sibilance és a rumble mérőket szándékos követési torzítással, illetve motorrezgéssel teszteltük az FFT és a szűrők matematikai érvényességét.

3.5.2. Teljesítmény-optimalizáció

A kezdeti tesztelés feltárta, hogy a hangszedő statisztikáinak másodpercenkénti írása súlyos I/O szűk keresztmetszeteket okozott. Ezt az SQLite Write-Ahead Logging (WAL) módjának engedélyezésével és egy memóriabufferelő rendszer implementálásával oldottuk meg. A rendszer az értékeket a memóriában halmozza fel, és csak 10 másodpercenként írja ki a fizikai tárhelyre, ami drasztikusan csökkentette a CPU terhelését egy Raspberry Pi környezetben.

3.6. Mesterséges intelligencia alkalmazása a fejlesztés során

A projekt megvalósítása során modern nagynyelvi modellek (LLM) alkalmazása történt, amelyek elsődlegesen segítő eszközként, a fejlesztési folyamat gyorsítása és optimalizálása érdekében szolgáltak. Az AI használata a következő területeken történt:

- **Koncepciók elsajátítása és kutatás:** Ismeretlen technológiai területek (például a digitális jelfeldolgozás specifikus matematikai műveletei vagy komplex API integrációk) esetén az AI-t mint gyors referenciát használtam. A kapott példakódokat nem implementáltam közvetlenül, hanem alapvető iránymutatónak tekintettem őket, amelyeket egy egyedi tervezési folyamat után adaptáltam a projekt specifikus igényeihez.
- **Ismétlődő kódstruktúrák (boilerplate) generálása:** A fejlesztés során felmerülő repetitív feladatok, mint például a frontend és backend közötti alapvető adatcsere logikája vagy a standard API végpontok vázlata, generálása során alkalmaztam AI segítséget. Ezek a kódrészletek egy alapvázlat szolgáltattak, amelyek végleges formáját manuális átírással és finomhangolással értem el.

- **Hibakeresés és optimalizálás:** A debugolási folyamat során az AI-t kódrészletek elemzésére és potenciális hibaforrások gyorsabb azonosítására használtam. Mivel az AI nem ismerte a teljes projekt architektúráját, a javasolt megoldásokat kritikus szemlélettel szűrtem, és csak a projekt kontextusába illeszthető megoldásokat integráltam a végleges kódbázisba.

Külön kiemelném, hogy a rendszer architektúrájának tervezése, a specifikus matematikai logikák illesztése, valamint a végleges szoftveres implementáció teljes egészében a szerző munkájának eredménye. Az AI használata kizárólag a fejlesztési ciklus hatékonyságának növelésére és a tanulási folyamat gyorsítására szolgált.

4. fejezet

Összegzés és jövőbeli fejlesztések

4.1. Az elért eredmények értékelése

A Smart Turntable Assistant sikeresen eléri elsődleges célját, az analóg bakelit-lemez-hallgatási élmény modernizálását. A nyers elektromos audiojel passzív monitorozásával a rendszer áthidalja a fizikai média és a digitális ökoszisztéma közötti szakadékot. Az alkalmazás közel 80% pontossággal azonosítja a lejátszott sávokat, a hallgatási munkamenetet nagy felbontású borítóképekkel és szinkronizált dalszövegekkel gazdagítja, és zökkenőmentesen naplózza a hallgatási előzményeket a külső közösségi zenei platformokra.

Továbbá, az audiofil szintű diagnosztikai eszközök beépítése valós idejű vizuális visszajelzést biztosít a felhasználóknak a tű és a lemezek fizikai állapotáról. A rendkívül moduláris, szétválasztott (decoupled) architektúra biztosítja, hogy a mögöttes Python backend és a reaktív webes interfész hatékonyan működjön egy olyan professzionális és robusztus szoftveres megoldásban, amely tiszteletben tartja a hagyományos hanglemez élményt, miközben modern digitális kényelmet nyújt.

4.2. Jövőbeli fejlesztési lehetőségek

Bár a jelenlegi implementáció teljesíti az összes kezdeti követelményt, számos lehetőség kínálkozik a jövőbeli továbbfejlesztésre. A legjelentősebb fejlődés saját neuron hálók használatával való sibilance detection és két zeneszám közti különbség (tempó és hangszín) detektálásával lenne elérhető.

Ezenfelül a diagnosztikai csomag kibővíthető lenne egy automatizált lemezjátszó-kalibrációs móddal is. Egy lemez lejátszásával ki tudja választani a felhasználó, hogy hol megy már a zene, ezzel a music detection algoritmus érzékenységét egyből be tudja állítani egy elfogadható állapotra, minimalizálva a manuális konfigurációt.

Ábrák jegyzéke

3.1. UML Use Case Diagram a felhasználó és a rendszer interakcióiról. . .	10
3.2. Deployment diagram, amely a Docker image felépítését mutatja be. . .	11
3.3. Entity-Relationship Diagram az adatbázis sémájáról.	12
3.4. UML Sequence Diagram a többszálú adatfeldolgozásról.	13